

CFGS Desenvolupament d'Aplicacions Multiplataforma
CFGS Desenvolupament d'Aplicacions WEB
0485_ICB0B2 – Programació II
EAC8
(Curs 2025-26 / 2n semestre)

Taula de continguts

Presentació i resultats d'aprenentatge	2
Criteris d'avaluació de l'EAC	2
Forma i dates de lliurament	3
Introducció	3
Descripció del funcionament	4
Estructura del projecte	7
Classes ja existents	8
Classe UtilsIO	8
Arxius per la configuració	9
Arxiu pom.xml	9
Arxiu hibernate.cfg.xml	9
Classes a generar	10
Classe HibernateUtil.java	10
Classe Bet.java	12
Classe Event.java	16
Classe BetDAO.java	21
Classe EventDAO.java	24
Criteris de correcció i ponderació	28

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 1 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Presentació i resultats d'aprenentatge

Aquest exercici d'avaluació continuada (EAC) es correspon amb el resultat d'aprenentatge RA8: "Utilitza bases de dades orientades a objectes, analitzant-ne les característiques i aplicant tècniques per mantenir la persistència de la informació." i RA9 "Gestiona informació emmagatzemada en bases de dades mantenint la integritat i consistència de les dades".

Els criteris d'avaluació corresponents són:

RA8:

1. Identifica les característiques de les bases de dades orientades a objectes.
2. Analitza l'aplicació al desenvolupament d'aplicacions mitjançant llenguatges orientats a objectes.
3. Instal·la sistemes gestors de bases de dades orientats a objectes.
4. Classifica i analitza els diferents mètodes suportats pels sistemes gestors per a la gestió de la informació emmagatzemada.
5. Crea bases de dades i les estructures necessàries per al emmagatzematge d'objectes.
6. Programa aplicacions que emmagatzemen objectes a les bases de dades creades.
7. Realitza programes per recuperar, actualitzar i eliminar objectes de les bases de dades.
8. Realitza programes per emmagatzemar i gestionar tipus de dades estructurades, compostes i relacionades.

RA9:

1. Identifica les característiques i els mètodes d'accés a sistemes gestors de bases de dades.
2. Programa connexions amb bases de dades.
3. Escriu codi per a emmagatzemar informació en bases de dades.
4. Crea programes per recuperar i mostrar informació emmagatzemada en bases de dades.
5. Efectua esborraments i modificacions sobre la informació emmagatzemada.
6. Crea aplicacions que mostrin la informació emmagatzemada en bases de dades.
7. Crea aplicacions per gestionar la informació present en bases de dades.

Criteris d'avaluació de l'EAC

La puntuació màxima assignada a cada activitat s'indica a l'enunciat.

Els criteris que es tindran en compte per avaluar el treball de l'alumnat són els següents:

- Que faci el que es demana.
- Que la descomposició dels problemes sigui correcta i adequada.
 - La descomposició contempli la resolució completa del problema plantejat
 - Tingui un nivell de detall (profunditat i descripció) adequat.
- Que s'hagin corregit els errors de compilació i funcioni.
- Que sigui codi ordenat, clar i fàcil de llegir:
 - Tot codi ha de tenir una capçalera apropiada.
 - Tot el codi ha d'estar correctament indentat (tabulació adequada).
 - Cal posar noms significatius a les variables i mètodes.
- Fer el lliurament en format .zip.
- Fer servir el projecte adjunt per a construir la solució.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 2 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Forma i dates de lliurament

Adjunt a aquest enunciat hi ha un projecte amb el nom AppEAC8 que haureu de descarregar, i **sobre el que haureu de programar** la vostra proposta de solució. **No s'avaluarà una solució que vingui en un altre format.** Podeu trobar més informació sobre el format del projecte i els IDE recomanats en aquest vídeo.

Es lliurarà **tot el projecte**, incorporant tot el vostre codi, i comprimit en un arxiu *zip* o *7z*. El nom del fitxer comprimit que contingui la solució serà el següent: **DA2_M03B2_EAC8_Cognom1_Inicial del cognom2**. Els cognoms s'escriuran sense accents. Per exemple, l'estudiant *Joan García Santos* posaria el següent nom al seu fitxer: **DA2_M03B2_EAC8_Garcia_S**.

L'arxiu s'ha d'enviar la solució a la bústia de **"M03B2 Lliurament EAC8"** de l'aula, dins del termini establert. Tingueu en compte que el sistema no permetrà fer lliuraments després de la data i hora indicades.

El termini de lliurament de l'**EAC8** finalitzarà a les **23:55 h** del dia **12/05/2026**. La proposta de solució és publicarà el dia **13/05/2026**. Les qualificacions es mostraran el dia **20/05/2026**.

Introducció

Com a última part d'aquest aprenentatge, veurem la persistència mitjançant base de dades. Aquesta aproximació és més complexa, però molt més semblant a la que es pot trobar a qualsevol empresa.

Les tecnologies que es treballaran en aquest EAC són:

- Base de dades: PostgreSQL.
- Llenguatge de programació: JAVA.
- Eina ORM (object relational mapping): Hibernate.
- Gestor de paquets JAVA: Maven.

Nota important: al campus hi ha disponible un **tutorial detallat** per poder instal·lar tot l'entorn necessari per portar a terme la pràctica de manera satisfactòria.

Nota 2: La principal diferència amb els EACs anterior és:

1. La persistència es portarà a terme mitjançant una base de dades PostgreSQL.
2. Haurà de fer-se servir Maven per instal·lar les dependències.
3. Haurà de fer-se servir Hibernate per a generar de manera automàtica la taula, amb les seves columnes i índexs.

Nota 3: A l'hora de configurar les variables de l'entorn, les següents són **obligatòries**, per tal que el professor pugui testar el projecte:

- Host de la base de dades: localhost
- Port de la base de dades: 5432
- Usuari de la base de dades: hibuser
- Password de la base de dades: password

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 3 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Descripció del funcionament

El següent vídeo descriu amb detall el funcionament del programa, els missatges de comunicació amb l'usuari i el control dels errors. S'haurà de replicar el funcionament de manera exacta.

A continuació es mostren les captures de pantalla de tots els menús que han d'implementar-se.

```
-----  
BET IOC!  
-----  
1) Afegir una esdeveniment.  
2) Afegir una aposta a un esdeveniment.  
3) Veure el llistat d'apostes.  
0) Sortir.  
  
Introdueixi un valor enter per l'opció
```

Figura 1: Menú inicial

```
Introdueixi el tipus d'esdeveniment  
Petanca  
Introdueixi el nom de l'esdeveniment  
Vilanova - Cubelles  
Introdueixi la data de l'esdeveniment (format: yyyyMMdd)  
20270101  
Introdueixi l'hora (00-23)  
23  
Introdueixi els minuts (00-59)  
00  
Hibernate:  
insert  
into  
events  
(name, timestamp, type)  
values  
(?, ?, ?)  
  
INFO  
Esdeveniment afegit correctament (ID: 44)
```

Figura 2: Afegir un esdeveniment

```
Introdueixi el tipus d'esdeveniment  
Petanca  
Introdueixi el nom de l'esdeveniment  
Vilanova - Cubelles  
Introdueixi la data de l'esdeveniment (format: yyyyMMdd)  
20260101  
Introdueixi l'hora (00-23)  
23  
Introdueixi els minuts (00-59)  
00  
  
ERROR  
Error en afegir l'aposta. La marca de temps ha de ser en el futur
```

Figura 3: Error per data passada

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 4 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

```

Introdueixi el tipus d'esdeveniment
Petanca
Introdueixi el nom de l'esdeveniment
Vilanova - Cubelles
Introdueixi la data de l'esdeveniment (format: yyyyMMdd)
format_incorrecte
Introdueixi l'hora (00-23)
23
Introdueixi els minuts (00-59)
00

ERROR

Format de data invàlid

```

Figura 4: Error per format de la data

```

Introdueixi el tipus d'esdeveniment
Petanca
Introdueixi el nom de l'esdeveniment
Vilanova - Cubelles
Introdueixi la data de l'esdeveniment (format: yyyyMMdd)
20270101
Introdueixi l'hora (00-23)
25
Introdueixi els minuts (00-59)
00

ERROR

Format d'hora invàlid

```

Figura 5: Error per format de l'hora

```

Apr 03, 2026 3:41:13 PM org.hibernate.Version logVersion
INFO: HHH000412: Hibernate ORM core version 6.4.4.Final
Apr 03, 2026 3:41:13 PM org.hibernate.cache.internal.RegionFactoryInitiator initiateService
INFO: HHH000026: Second-level cache disabled
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl configure
WARN: HHH10001002: Using built-in connection pool (not intended for production use)
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001005: Loaded JDBC driver class: org.postgresql.Driver
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001012: Connecting with JDBC URL [jdbc:postgresql://localhost:5432/hibernatedb]
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: (password=****, user=hibuser)
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH10001115: Connection pool size: 20 (min=1)
Apr 03, 2026 3:41:13 PM org.hibernate.engine.jdbc.dialect.internal.DialectFactoryImpl constructDialect
WARN: HHH90000025: PostgreSQLDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be select
ed by default)
Apr 03, 2026 3:41:13 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcCo
nnectionAccess@4ea17147] for (non-JTA) DDL execution was not in auto-commit mode; the Connection 'local transaction' will be committed and the Connection
will be set into auto-commit mode.
Hibernate SessionFactory inicialitzat amb èxit
Hibernate:
select
  e1_0.id,
  e1_0.name,
  e1_0.timestamp,
  e1_0.type
from
  events e1_0
order by
  e1_0.type,
  e1_0.timestamp

Esdeveniments disponibles:
1) Real Madrid - Barcelona - Basketball - 2027-04-02T18:15:27.552110
2) Valencia Basket - Unicaja - Basketball - 2027-04-05T20:41:03.001248
3) Joventut - Baskonia - Basketball - 2027-04-08T11:09:36.908774
4) Gran Canaria - Murcia - Basketball - 2027-04-11T17:54:22.662390
5) Manresa - Zaragoza - Basketball - 2027-04-14T13:26:48.443109
6) Tenerife - Obradoiro - Basketball - 2027-04-17T19:38:59.777003
7) Breogan - Andorra - Basketball - 2027-04-20T10:51:14.118900
8) Estudiantes - Fuenlabrada - Basketball - 2027-04-23T21:03:41.556012
9) Barcelona - Baskonia - Basketball - 2027-04-26T16:29:07.990122
10) Valencia Basket - Real Madrid - Basketball - 2027-04-29T18:44:53.337701
11) Vilanova - Cubelles - Petanca - 2027-01-01T23:00

```

Figura 6:

Afegir aposta. Seleccionar esdeveniment.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 5 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

```
Seleccioni un esdeveniment (1-41)
11
Introdueixi el nom de l'apostant
11111111H
Introdueixi la descripció de l'aposta
Guanya Vilanova
Introdueixi les quotes de l'aposta
1.4
Introdueixi l'import de l'aposta
200
Hibernate:
select
  e1_0.id,
  e1_0.name,
  e1_0.timestamp,
  e1_0.type
from
  events e1_0
where
  e1_0.id=?
Hibernate:
insert
into
  bets
(amount, bet_description, bettor_name, event_id, odds)
values
  (?, ?, ?, ?, ?)

-----
INFO
Aposta afegida correctament
```

Figura 7: Afegir aposta

```
Seleccioni un esdeveniment (1-41)
43
-----
ERROR
No s'ha introduït una opció vàlida
```

Figura 8: Esdeveniment erroni

Introdueixi un valor enter per l'opció
3
Introdueixi el tipus d'esdeveniment

Figura 9: Veure apostes (tipus buit)

DATETIME	SPORT	ESDEVENIMENT	JUGADOR	TIPUS	QUOTES	IMPORT
2027040218	Basketball	Real Madrid - Barc	10000112B	Real Madrid -7.5 handicap	1.85	120.00
2027040218	Basketball	Real Madrid - Barc	10000043B	Real Madrid win	1.85	120.00
2027040218	Basketball	Real Madrid - Barc	10000110D	Under 160.5 points	2.10	80.00
2027040218	Basketball	Real Madrid - Barc	10000109P	Over 165.5 points	1.90	95.00
2027040218	Basketball	Real Madrid - Barc	10000111X	Barcelona win	2.05	100.00
2027040218	Basketball	Real Madrid - Barc	10000011X	Real Madrid win	1.80	140.00
2027040520	Basketball	Valencia Basket -	10000012B	Over 160.5 points	1.95	100.00
2027040520	Basketball	Valencia Basket -	10000123F	Valencia -6.5 handicap	1.90	130.00
2027040520	Basketball	Valencia Basket -	10000122Y	Unicaja win	2.35	70.00
2027040520	Basketball	Valencia Basket -	10000121M	Under 155.5 points	1.85	110.00
2027040520	Basketball	Valencia Basket -	10000124P	Over 162.5 points	2.05	85.00
2027040811	Basketball	Juventut - Baskoni	10000013N	Baskonia win	2.20	85.00
2027041117	Basketball	Gran Canaria - Mur	10000014J	Gran Canaria win	1.65	160.00
2027041413	Basketball	Manresa - Zaragoza	10000015Z	Manresa win	1.90	70.00
2027041719	Basketball	Tenerife - Obradoi	10000016S	Tenerife win	1.55	180.00
2027042010	Basketball	Breogan - Andorra	10000017Q	Under 158.5 points	2.05	95.00
2027042321	Basketball	Estudiantes - Fuen	10000018W	Estudiantes win	1.75	125.00
2027042616	Basketball	Barcelona - Baskon	10000019A	Barcelona win	1.60	210.00
2027042918	Basketball	Valencia Basket -	10000020G	Real Madrid -5.5 handicap	2.30	90.00
2027010123	Petanca	Vilanova - Cubelle	11111111H	Guanya Vilanova	1.40	200.00

Figura 10: Llistat

d'esdeveniments sense filtrar

```

Introdueixi un valor enter per l'opció
3
Introdueixi el tipus d'esdeveniment
pet
Hibernate:
select
  e1_0.id,
  e1_0.name,
  e1_0.timestamp,
  e1_0.type
from
  events e1_0
where
  e1_0.type ilike ? escape ''
order by
  e1_0.timestamp
Hibernate:
select
  b1_0.id,
  b1_0.amount,
  b1_0.bet_description,
  b1_0.bettor_name,
  b1_0.event_id,
  b1_0.odds
from
  bets b1_0
where
  b1_0.event_id=?

```

DATETIME	SPORT	ESDEVENIMENT	JUGADOR	TIPUS	QUOTES	IMPORT
2027010123	Petanca	Vilanova - Cubelle	11111111H	Guanya Vilanova	1.40	200.00

Figura 11: Llistat

d'apostes filtrat

Aquest funcionament ha d'implementar-se a la classe App.java.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 7 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Estructura del projecte

Aquest projecte constarà de 6 classes Java (ja creades al material adjunt):

- **Application.java:** Contindrà el programa principal, que crearà instàncies de les altres classes i gestionarà la interacció amb l'usuari.
- **Paquet "util"**
 - **Constants.java:** Contindrà totes les constants necessàries per al programa, com ara noms de fitxers, missatges d'error, i formats de dades.
 - **UtilsIO.java:** Contindrà totes les operacions d'entrada/sortida per a la interacció amb l'usuari, com ara mostrar menús, demanar dades i mostrar errors o informació.
 - **HibernateUtil.java:** Configurarà i gestionarà la connexió amb Hibernate (SessionFactory i transaccions).
- **Paquet "model"**
 - **Bet.java:** Representarà una aposta soci individual amb les seves dades bàsiques (nom, descripció, quotes, quantitat).
 - **Event.java:** Representarà un esdeveniment individual amb les seves dades bàsiques (tipus, nom, data).
- **Paquet "dao"**
 - **BetDAO.java:** Gestionarà l'accés a la base de dades per les apostes (operacions CRUD mitjançant Hibernate).
 - **EventDAO.java:** Gestionarà l'accés a la base de dades pels esdeveniments (operacions CRUD mitjançant Hibernate).

A continuació es mostra l'estructura del projecte, amb les diferents carpetes i classes.



Figura 12: Estructura del projecte

Totes les classes estan creades. En concret, UtilsIO ho està a la seva totalitat. La resta de classes han de desenvolupar-se.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 8 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

A més, existeixen dos arxius de configuració amb format .xml:

- hibernate.cfg.xml: conté la configuració d'hibernate amb, entre d'altres, les variables per a configurar la connexió amb la base de dades.
- pom.xml: arxiu maven amb la configuració de tot el projecte

Tal com s'ha dit anteriorment, hi ha un tutorial al campus, on pot trobar-se una configuració exemple d'un projecte amb PostgreSQL i Hibernate.

Classes ja existents

En aquest projecte es parteix de classes ja generades per l'equip, que **podran utilitzar-se sense necessitat de modificar-les**. A continuació es descriuen.

Classe UtilsIO

Aquesta classe proporciona mètodes útils per a la interacció amb l'usuari i la presentació de dades.

Mètodes principals:

- showAnyMessage(String header, String mainText): Mostra un missatge amb capçalera i text principal.
- showMenu(String menuText): Mostra un menú.
- showError(String errorText): Mostra un missatge d'error.
- showInfo(String infoText): Mostra un missatge informatiu.
- askForAnyString(String message): Demana a l'usuari que introdueixi una cadena de text.
- askForNotEmptyString(String message, String errorMessage): Demana a l'usuari que introdueixi una cadena de text no buida.
- askForInteger(String message, String errorMessage): Demana a l'usuari que introdueixi un nombre enter.
- askForFloat(String message, String errorMessage): Demana a l'usuari que introdueixi un nombre decimal.
- showBets(String betList): Mostra una llista d'apostes formatada a partir d'un String, validant l'estructura i ignorant entrades incorrectes.

Aquesta classe facilita la creació d'interfícies d'usuari per consola, gestionant l'entrada i sortida de dades de manera consistent i robusta. Podeu utilitzar aquests mètodes per interactuar amb l'usuari sense haver de preocupar-vos pels detalls d'implementació de la interfície.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 9 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Arxius per la configuració

Tal com s'ha parlat anteriorment, haurà de generar-se dos arxius per la configuració: pom.xml i hibernate.cfg.xml.

Nota: aquests dos arxius poden copiar-se a la seva totalitat del tutorial que pot trobar-se al campus.

Arxiu pom.xml

Funció principal: Fitxer de configuració de Maven que defineix l'estructura i dependències del projecte de gestió de socis. Gestiona la compilació amb Java 21 i les dependències necessàries per a persistència, testing i logging.

Dependències principals:

- Hibernate ORM (6.4.4) i PostgreSQL JDBC (42.7.2) per a persistència a base de dades
- JUnit 5 (5.11.0) per a testing
- SLF4J (2.0.9) per a logging
- Jakarta Persistence API (3.1.0) com a implementació de JPA

Arxiu hibernate.cfg.xml

Funció principal: Fitxer de configuració de Hibernate que estableix la connexió a PostgreSQL i defineix el comportament de l'ORM. Configura la base de dades local "hibernatedb" amb usuari "hibuser".

Paràmetres clau:

- Connexió: Driver PostgreSQL, URL localhost:5432, dialecte específic
- Comportament: Mostra SQL formatat, actualitza esquema automàticament
- Mapeig: Inclou l'entitat Member del paquet cat.exemple.model

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 10 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Classes a generar

A continuació es presenten les classes a generar.

Classe HibernateUtil.java

Nota: Aquesta classe és una utilitat per a la configuració i gestió de la connexió amb Hibernate, patró Singleton que garanteix una única instància de SessionFactory a tota l'aplicació. Pot copiar-se directament del tutorial que es troba al campus.

Descripció: La classe HibernateUtil s'encarrega de la inicialització i configuració de Hibernate, creant i gestionant el SessionFactory que s'utilitzarà a tot el projecte per a l'accés a la base de dades. Implementa el patró Singleton per assegurar que només hi hagi una instància de SessionFactory.

Propietats

1. sessionFactory (SessionFactory, estàtic i final): L'únic punt d'accés a les funcionalitats de Hibernate. S'inicialitza una sola vegada durant la càrrega de la classe.

Mètodes**Bloc d'inicialització estàtic**

1. Funció: S'executa una sola vegada quan la classe es carrega a memòria
2. Responsabilitats:
 1. Crea una nova instància de Configuration
 2. Carrega la configuració des de hibernate.cfg.xml
 3. Registra les classes del model (Event i Bet)
 4. Construeix el ServiceRegistry amb les propietats de configuració
 5. Inicialitza el SessionFactory
 6. Gestiona errors durant la inicialització
3. Control d'errors:
 - En cas d'error durant la inicialització, captura l'excepció
 - Mostra l'error per consola
 - Llança ExceptionInInitializerError per evitar que l'aplicació continuï amb una configuració incorrecta
4. Missatges:
 - "Hibernate SessionFactory inicialitzat amb èxit" en cas d'èxit
 - "Error en la inicialització de SessionFactory: " + ex en cas d'error

getSessionFactory

1. Signatura: public static SessionFactory getSessionFactory()
2. Retorn: La instància única de SessionFactory
3. Funció: Proporciona accés global al SessionFactory inicialitzat
4. Ús: SessionFactory factory = HibernateUtil.getSessionFactory();

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 11 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

shutdown

1. Signatura: `public static void shutdown()`
2. Funció: Tanca el SessionFactory i allibera tots els recursos
3. Comportament:
 1. Verifica que el SessionFactory existeixi i no estigui ja tancat
 2. Tanca el SessionFactory
 3. Mostra missatge "SessionFactory tancat" per confirmació
4. Ús: `HibernateUtil.shutdown();`

Configuració

La classe espera trobar:

1. Un arxiu `hibernate.cfg.xml` al classpath amb la configuració de Hibernate
2. Les classes del model anotades correctament amb `@Entity`

Patró de Disseny

Implementa el patró Singleton mitjançant:

1. Constructor implícitament privat (no instanciable)
2. Variable static final que garanteix una única instància
3. Mètode static per accedir a la instància

Cicle de Vida

1. Inicialització: Durant la càrrega de la classe
2. Ús: Mitjançant `getSessionFactory()`
3. Finalització: Explicitament amb `shutdown()` (recomanat al tancar l'aplicació)

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 12 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Classe Bet.java

Nota: Aquesta classe representa una aposta dins del sistema, mapejada com a entitat JPA i associada a un esdeveniment (Event). Forma part del model de dades persistent i encapsula la informació i validacions bàsiques d'una aposta.

Descripció: Modela una aposta realitzada per un usuari sobre un esdeveniment esportiu. Inclou dades com el nom de l'apostant, la descripció de l'aposta, les quotes i l'import apostat. La classe incorpora validacions en el constructor i estableix una relació many-to-one amb l'entitat Event.

Anotacions JPA

- **@Entity:** Marca la classe com una entitat persistent.
- **@Table(name = "bets"):** Especifica el nom de la taula a la base de dades.
- **@Id:** Defineix la clau primària de l'entitat.
- **@GeneratedValue(strategy = GenerationType.IDENTITY):** Configura la generació automàtica de l'identificador.
- **@Column:** Defineix les propietats de les columnes (nom, nul·labilitat).
- **@ManyToOne(fetch = FetchType.LAZY):** Defineix una relació many-to-one amb càrrega diferida.
- **@JoinColumn(name = "event_id"):** Especifica la columna de la clau forana associada a Event.

Propietats

1. **id (Long):** Identificador únic de l'aposta generat automàticament. Clau primària.
2. **bettorName (String):** Nom de l'apostant. No pot ser nul ni buit.
3. **betDescription (String):** Descripció de l'aposta (ex: "Over 2.5 goals"). No pot ser nul ni buit.
4. **odds (float):** Quotes de l'aposta. Han de ser majors que 0.
5. **amount (float):** Import apostat. Ha de ser major que 0.
6. **event (Event):** Esdeveniment associat a l'aposta. Relació many-to-one.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 13 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Mètodes

Constructor per defecte

1. **Signatura:** public Bet()
2. **Funció:** Constructor buit requerit per JPA.
3. **Ús:** Utilitzat pel framework per instanciar l'entitat des de la base de dades.
4. **Exemple:** Bet b = new Bet();

Constructor parametritzat

1. **Signatura:**

public Bet(String bettorName, String betDescription, float odds, float amount)

2. **Paràmetres:**

- bettorName: Nom de l'apostant (no pot ser null ni buit)
- betDescription: Descripció de l'aposta (no pot ser null ni buit)
- odds: Quotes (> 0)
- amount: Import apostat (> 0)

3. **Funció:** Crea una aposta amb validacions completes dels camps obligatoris.

4. **Validacions:**

- Llança IllegalArgumentException si:
 - bettorName és null o buit
 - betDescription és null o buit
 - odds ≤ 0
 - amount ≤ 0

5. **Exemple:** Bet b = new Bet("Jordi", "Osasuna win", 2.5f, 50.0f);

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 14 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Getters**getId**

1. **Signatura:** public Long getId()
2. **Retorn:** Identificador de l'aposta (pot ser null si no està persistida).
3. **Exemple:** Long id = bet.getId();

getBettorName

1. **Signatura:** public String getBettorName()
2. **Retorn:** Nom de l'apostant.
3. **Exemple:** String name = bet.getBettorName();

getBetDescription

1. **Signatura:** public String getBetDescription()
2. **Retorn:** Descripció de l'aposta.
3. **Exemple:** String desc = bet.getBetDescription();

getOdds

1. **Signatura:** public float getOdds()
2. **Retorn:** Quotes de l'aposta.
3. **Exemple:** float odds = bet.getOdds();

getAmount

1. **Signatura:** public float getAmount()
2. **Retorn:** Import apostat.
3. **Exemple:** float amount = bet.getAmount();

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 15 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

getEvent

1. **Signatura:** public Event getEvent()
2. **Retorn:** Esdeveniment associat.
3. **Consideració:** Pot provocar càrrega lazy.
4. **Exemple:** Event e = bet.getEvent();

Setters**setId**

1. **Signatura:** public void setId(Long id)
2. **Paràmetre:** id – Identificador de l'aposta
3. **Funció:** Assigna l'identificador (normalment gestionat per JPA).
4. **Exemple:** bet.setId(1L);

setEvent

1. **Signatura:** public void setEvent(Event event)
2. **Paràmetre:** event – Esdeveniment associat
3. **Funció:** Estableix la relació amb un Event.
4. **Consideracions:** Relació bidireccional gestionada habitualment des d'Event.
5. **Exemple:** bet.setEvent(event);

Mètodes funcionals**toString**

1. **Signatura:** public String toString()
2. **Retorn:** Cadena formatada amb les dades de l'aposta.
3. **Propòsit:** Representació textual per visualització o exportació.
4. **Format:** Definida per Constants.BET_TO_STRING_FORMAT.
5. **Exemple:** String text = bet.toString();

Retorna
"Jordi,Osasuna win,2.50,50.00"

(exemple):

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 16 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Classe Event.java

Nota: Aquesta classe representa un esdeveniment esportiu dins del sistema, mapejat com a entitat JPA i relacionat amb múltiples apostes (Bet). Inclou validacions de coherència temporal i gestiona tant la persistència com una capa de compatibilitat amb una API antiga basada en mapes.

Descripció: Modela un esdeveniment esportiu sobre el qual es poden realitzar apostes. Conté informació bàsica com el tipus d'esport, el nom de l'esdeveniment i la seva data/hora. Gestiona una relació one-to-many amb les apostes i proporciona mètodes per afegir-les i consultar-les de manera segura, incloent validacions d'entrada i coherència del model.

Anotacions JPA

- **@Entity:** Marca la classe com una entitat persistent.
- **@Table(name = "events"):** Especifica el nom de la taula a la base de dades.
- **@Id:** Defineix la clau primària.
- **@GeneratedValue(strategy = GenerationType.IDENTITY):** Configura la generació automàtica de l'identificador.
- **@Column:** Defineix les propietats de les columnes (nul·labilitat).
- **@OneToMany(mappedBy = "event", cascade = CascadeType.ALL, orphanRemoval = true):** Relació one-to-many amb Bet, amb persistència en cascada i eliminació d'orfes.
- **@Transient:** Indica que el camp no es persisteix a la base de dades.

Propietats

1. **id (Long):** Identificador únic de l'esdeveniment generat automàticament. Clau primària.
2. **type (String):** Tipus d'esport (ex: "Soccer", "Basketball"). No pot ser nul ni buit.
3. **name (String):** Nom o descripció de l'esdeveniment. No pot ser nul ni buit.
4. **timestamp (LocalDateTime):** Data i hora de l'esdeveniment. Ha de ser futura.
5. **betsList (List<Bet>):** Llista persistent d'apostes associades (relació one-to-many).
6. **bets (Map<String, Bet>):** Mapa transitori d'apostes per compatibilitat amb API antiga (no persistent).

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 17 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Mètodes

Constructor per defecte

1. **Signatura:** public Event()
2. **Funció:** Constructor buit requerit per JPA. Inicialitza el mapa transitori bets.
3. **Ús:** Instanciació per part del framework de persistència.
4. **Exemple:** Event e = new Event();

Constructor parametritzat

1. Signatura:

public Event(String type, String name, LocalDateTime timestamp)

2. Paràmetres:

- type: Tipus d'esport (no pot ser null ni buit)
- name: Nom de l'esdeveniment (no pot ser null ni buit)
- timestamp: Data/hora (no pot ser null i ha de ser futura)

3. Funció:

Crea un esdeveniment amb validacions completes dels camps obligatoris.

4. Validacions:

- Llança IllegalArgumentException si:
 - type o name són null o buits
 - timestamp és null
 - timestamp no és posterior al moment actual

5. Exemple:

Event e = new Event("Soccer", "Barça vs Madrid", LocalDateTime.now().plusDays(1));

Getters

getId

1. **Signatura:** public Long getId()
2. **Retorn:** Identificador de l'esdeveniment (pot ser null si no està persistit).
3. **Exemple:** Long id = event.getId();

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 18 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

getType

1. **Signatura:** public String getType()
2. **Retorn:** Tipus d'esport.
3. **Exemple:** String type = event.getType();

getName

1. **Signatura:** public String getName()
2. **Retorn:** Nom de l'esdeveniment.
3. **Exemple:** String name = event.getName();

getTimestamp

1. **Signatura:** public LocalDateTime getTimestamp()
2. **Retorn:** Data i hora de l'esdeveniment.
3. **Exemple:** LocalDateTime t = event.getTimestamp();

Setters**setId**

1. **Signatura:** public void setId(Long id)
2. **Paràmetre:** id – Identificador de l'esdeveniment
3. **Funció:** Assigna l'identificador (habitualment gestionat per JPA).
4. **Exemple:** event.setId(1L);

setType

1. **Signatura:** public void setType(String type)
2. **Paràmetre:** type – Tipus d'esport
3. **Funció:** Actualitza el tipus d'esport.
4. **Validacions:** Llança IllegalArgumentException si és null o buit.
5. **Exemple:** event.setType("Tennis");

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 19 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

setName

1. **Signatura:** public void setName(String name)
2. **Paràmetre:** name – Nom de l'esdeveniment
3. **Funció:** Actualitza el nom.
4. **Validacions:** Llança IllegalArgumentException si és null o buit.
5. **Exemple:** event.setName("Nadal vs Djokovic");

setTimestamp

1. **Signatura:** public void setTimestamp(LocalDateTime timestamp)
2. **Paràmetre:** timestamp – Nova data/hora
3. **Funció:** Actualitza la data de l'esdeveniment.
4. **Validacions:**
 - No pot ser null
 - Ha de ser futura
5. **Exemple:** event.setTimestamp(LocalDateTime.now().plusDays(2));

Mètodes funcionals**addBet**

1. **Signatura:** public void addBet(String betDescription, String bettor, float odds, float amount)
2. **Paràmetres:** Descripció, apostant, quotes i import
3. **Propòsit:** Crea i associa una nova aposta a l'esdeveniment.
4. **Funcionament:**
 - Valida dades d'entrada
 - Evita duplicats per apostant
 - Crea un Bet
 - Estableix la relació bidireccional
 - Afegeix a bets i betsList
5. **Exemple:** event.addBet("Over 2.5", "Jordi", 2.1f, 50f);

getBet

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 20 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

1. **Signatura:** `public Bet getBet(String bettor)`
2. **Retorn:** Aposta associada a l'apostant
3. **Validacions:** Llança excepció si no existeix o és buit
4. **Exemple:** `Bet b = event.getBet("Jordi");`

bettorExists

1. **Signatura:** `public boolean bettorExists(String bettor)`
2. **Retorn:** Indica si un apostant té aposta
3. **Exemple:** `boolean exists = event.bettorExists("Jordi");`

getAllBets

1. **Signatura:** `public Map<String, Bet> getAllBets()`
2. **Retorn:** Còpia defensiva del mapa d'apostes
3. **Propòsit:** Evitar modificacions externes
4. **Exemple:** `Map<String, Bet> bets = event.getAllBets();`

betsToString

1. **Signatura:** `public String betsToString()`
2. **Retorn:** Cadena amb totes les apostes separades per salts de línia
3. **Propòsit:** Representació textual de les apostes
4. **Exemple:** `String text = event.betsToString();`

eventToString

1. **Signatura:** `public String eventToString()`
2. **Retorn:** Representació CSV de l'esdeveniment (type,name,timestamp)
3. **Exemple:** `String csv = event.eventToString();`

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 21 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

toString

1. **Signatura:** public String toString()
2. **Retorn:** Representació completa de l'esdeveniment amb totes les apostes
3. **Propòsit:** Exportació o visualització combinada
4. **Funcionament:**
 - Combina dades de l'esdeveniment i cada aposta
 - Una línia per aposta
5. **Exemple:** String text = event.toString();

Classe BetDAO.java

Nota: Aquesta classe implementa el patró Data Access Object (DAO) per a l'entitat Bet, encapsulant l'accés a la base de dades mitjançant Hibernate. Gestiona explícitament sessions i transaccions per a cada operació.

Descripció: Proporciona operacions CRUD (Create, Read, Update, Delete) i consultes específiques sobre l'entitat Bet. Cada mètode obre una sessió Hibernate independent, gestiona la transacció corresponent i assegura la consistència de dades amb mecanismes de commit i rollback.

Anotacions / Infraestructura

- No és una entitat JPA (@Entity), sinó una classe de persistència.
- Utilitza HibernateUtil per obtenir sessions (SessionFactory).
- Fa ús de:
 - Session per operacions amb la base de dades
 - Transaction per control transaccional
 - Query per execució de consultes HQL
- Les consultes es defineixen a Constants (desacoblament de la capa DAO).

Propietats

Aquesta classe no té atributs d'instància.

👉 És una classe **stateless**, tota la lògica es troba als mètodes.

Mètodes d'accés (Lectura)

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 22 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

getById

1. **Signatura:** public Bet getById(Long id)
2. **Paràmetre:** id – Identificador de l'aposta
3. **Retorn:** Objecte Bet o null si no existeix
4. **Funció:** Recupera una aposta per ID mitjançant session.get()
5. **Exemple:** Bet bet = dao.getById(1L);

getAll

1. **Signatura:** public List<Bet> getAll()
2. **Retorn:** Llista de totes les apostes
3. **Funció:** Executa una consulta que retorna tots els registres.
4. **Exemple:** List<Bet> bets = dao.getAll();

findByBettorName

1. **Signatura:** public List<Bet> findByBettorName(String bettorName)
2. **Paràmetre:** Fragment del nom de l'apostant
3. **Retorn:** Llista d'apostes coincidents
4. **Funció:** Cerca amb operador LIKE (coincidència parcial)
5. **Exemple:** List<Bet> bets = dao.findByBettorName("Jordi");

findByEventId

1. **Signatura:** public List<Bet> findByEventId(Long eventId)
2. **Paràmetre:** ID de l'esdeveniment
3. **Retorn:** Apostes associades a l'esdeveniment
4. **Funció:** Consulta per relació amb Event
5. **Exemple:** List<Bet> bets = dao.findByEventId(10L);

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 23 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Mètodes de modificació (Persistència)**save**

1. **Signatura:** public Long save(Bet bet)
2. **Paràmetre:** bet – Entitat a persistir
3. **Retorn:** ID generat
4. **Funció:**
 - Obre sessió
 - Inicia transacció
 - Guarda l'objecte
 - Fa commit
5. **Control d'errors:**
 - Rollback en cas d'error
 - Llança excepció
6. **Exemple:** Long id = dao.save(bet);

update

1. **Signatura:** public void update(Bet bet)
2. **Paràmetre:** bet – Entitat existent
3. **Funció:** Actualitza l'entitat a la base de dades
4. **Control d'errors:**
 - Rollback en cas d'error
 - Llança excepció
5. **Exemple:** dao.update(bet);

delete

1. **Signatura:** public void delete(Bet bet)
2. **Paràmetre:** bet – Entitat a eliminar
3. **Funció:** Elimina l'entitat de la base de dades
4. **Control d'errors:**
 - Rollback en cas d'error

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 24 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

- Llança excepció

5. **Exemple:** dao.delete(bet);

Classe EventDAO.java

Nota: Aquesta classe implementa el patró Data Access Object (DAO) per a l'entitat Event, encapsulant totes les operacions d'accés a la base de dades mitjançant Hibernate. Gestiona el cicle de vida de sessions i transaccions de manera explícita.

Descripció: Proporciona operacions CRUD i consultes específiques sobre entitats Event. Cada mètode obre una sessió Hibernate independent, executa l'operació corresponent i gestiona la transacció associada, assegurant consistència i control d'errors. També permet recuperar esdeveniments segons criteris com el nom o el tipus.

Anotacions / Infraestructura

- No és una entitat JPA, sinó una classe DAO de persistència.
- Utilitza HibernateUtil per obtenir sessions (SessionFactory).
- Fa ús de:
 - Session per operacions amb la base de dades
 - Transaction per control transaccional
 - Query per execució de consultes HQL
- Les consultes es defineixen a Constants (desacoblament de la lògica SQL/HQL).

Propietats

Aquesta classe no defineix atributs d'instància.

📖 Classe **stateless**, orientada exclusivament a operacions de persistència.

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 25 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Mètodes d'accés (Lectura)**getById**

1. **Signatura:** public Event getById(Long id)
2. **Paràmetre:** id – Identificador de l'esdeveniment
3. **Retorn:** Objecte Event o null si no existeix
4. **Funció:** Recupera un esdeveniment per ID mitjançant session.get()
5. **Exemple:** Event event = dao.getById(1L);

getAll

1. **Signatura:** public List<Event> getAll()
2. **Retorn:** Llista de tots els esdeveniments
3. **Funció:** Executa una consulta HQL definida a Constants
4. **Exemple:** List<Event> events = dao.getAll();

findByName

1. **Signatura:** public List<Event> findByName(String name)
2. **Paràmetre:** Fragment del nom de l'esdeveniment
3. **Retorn:** Llista d'esdeveniments coincidents
4. **Funció:** Cerca amb operador LIKE (coincidència parcial)
5. **Exemple:** List<Event> events = dao.findByName("Soccer");

findByType

1. **Signatura:** public List<Event> findByType(String type)
2. **Paràmetre:** Tipus d'esport
3. **Retorn:** Llista d'esdeveniments coincidents
4. **Funció:** Cerca per tipus mitjançant consulta HQL amb paràmetre
5. **Exemple:** List<Event> events = dao.findByType("Basketball");

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 26 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Mètodes de modificació (Persistència)**save**

1. **Signatura:** public Long save(Event event)
2. **Paràmetre:** event – Entitat a persistir
3. **Retorn:** ID generat
4. **Funció:**
 - Obre sessió
 - Inicia transacció
 - Guarda l'objecte
 - Fa commit
5. **Control d'errors:**
 - Rollback en cas d'error
 - Llança excepció
6. **Exemple:** Long id = dao.save(event);

update

1. **Signatura:** public void update(Event event)
2. **Paràmetre:** event – Entitat existent
3. **Funció:** Actualitza l'esdeveniment a la base de dades
4. **Control d'errors:**
 - Rollback en cas d'error
 - Llança excepció
5. **Exemple:** dao.update(event);

delete

1. **Signatura:** public void delete(Event event)
2. **Paràmetre:** event – Entitat a eliminar
3. **Funció:** Elimina l'esdeveniment de la base de dades
4. **Consideracions:**
 - Elimina també les apostes associades (cascade = ALL)

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 27 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

5. Control d'errors:

- Rollback en cas d'error
- Llança excepció

6. Exemple: `dao.delete(event);`

	Codi: I71	Exercici d'avaluació contínua 8	Pàgina 28 de 29
	Versió: 03	0485_ICB0B2_EAC8_Enunciat_2526S2	Lliurament límit: 12/05

Criteris de correcció i ponderació

A continuació es mostren els criteris de correcció agrupats en 2 aspectes:

- Configuració correcta de l'entorn (5 punts). Es genera un projecte que utilitza el llenguatge Java i l'ORM Hibernate. Aquest projecte és capaç de crear una base de dades de manera correcta, amb una taula pels socis, amb els seus atributs com a columnes.
- Funcionament de l'aplicació (5 punts). El programa funciona de la mateixa manera descrita al vídeo de l'apartat Descripció del funcionament.

Configuració correcta de l'entorn (5 punts)

Nivell	Descripció
A	La configuració és correcta i totalment adaptada a les especificacions.
B	La configuració és correcta, però no adaptada a la seva totalitat a les especificacions.
C	La configuració no és correcta

Funcionament de l'aplicació (5 punts)

Nivell	Descripció
A	La funció principal compleix totalment els requisits especificats. Funciona sense errors, cobreix tots els casos possibles i ha estat provat exhaustivament.
B	El programa funciona correctament en la majoria dels casos, però pot contenir algun cas límit no gestionat adequadament. Errors mínims en execució.
C	El programa cobreix els requisits bàsics, però presenta errors d'execució que afecten parcialment la funcionalitat. Alguns casos no estan contemplats.
D	El programa té múltiples errors i no arriba a cobrir tots els requeriments. L'execució falla en diversos punts importants.
E	El codi no funciona o no cobreix la majoria dels requeriments de l'enunciat. No es pot executar o no fa res del que es demana.